

PLATAFORMA DE MONITOREO Y RIESGO PARA AGRICULTURA DE
PRESICIÓN (“AGROSMART”)

JESUS DAVID LEAL ESPINEL

C.C: 1094047490

INGENIERIA DEL SOFTWARE 1

UNIVERSIDAD DE PAMPLONA SEDE VILLA DEL ROSARIO

FACULTAD DE INGENIERIAS Y ARQUITECTURA

2026

1. Requerimientos de software

Documento SRS

Propósito: Desarrollar un sistema centralizado para recolectar telemetría de sensores en campo y automatizar el riego de cultivos mediante reglas de negocio e integración meteorológica.

Alcance: Abarca la recepción de datos IoT (humedad, temperatura), visualización en tiempo real, configuración de umbrales por zonas, control remoto de bombas de agua y generación de reportes.

Actor	Descripción
Ingeniero Agrónomo	Analiza datos, crea zonas de cultivo y configura umbrales automáticos.
Operario / Agricultor	Visualiza el estado actual y fuerza el riego de forma manual.
Sistema IoT (Hardware)	Envía datos de sensores y recibe comandos para accionar válvulas.

Requerimientos Funcionales (RF):

- **RF01 Ingesta de Telemetría:** Recibir y almacenar datos (humedad, temperatura) cada 5 minutos vía protocolo MQTT.
- **RF02 Gestión de Zonas:** Permitir agrupar sensores en "Zonas de Cultivo" representadas en un mapa.
- **RF03 Riego Automático:** Disparar encendido de bombas si la humedad cae por debajo del umbral definido.
- **RF04 Condición Meteorológica:** Bloquear el riego automático si la API externa pronostica lluvia en las próximas 12 horas.

- **RF05 Riego Manual:** Permitir al operario forzar el encendido/apagado de válvulas desde la interfaz web.
- **RF06 Alertas de Hardware:** Notificar por SMS/Correo si un sensor deja de emitir datos por más de 1 hora.
- **RF07 Dashboard en Tiempo Real:** Mostrar indicadores visuales actualizados sin necesidad de recargar la página.
- **RF08 Historial Gráfico:** Mostrar gráficas de líneas con la evolución de la temperatura y humedad en el último mes.
- **RF09 Configuración de Umbrales:** Permitir modificar los porcentajes mínimos/máximos de humedad por cada zona.
- **RF10 Reporte Agronómico:** Exportar los datos históricos en formato CSV y PDF.
- **RF11 Gestión de Usuarios:** Registro, login y asignación de roles (Operario vs. Agrónomo).

Requerimientos No Funcionales (RNF):

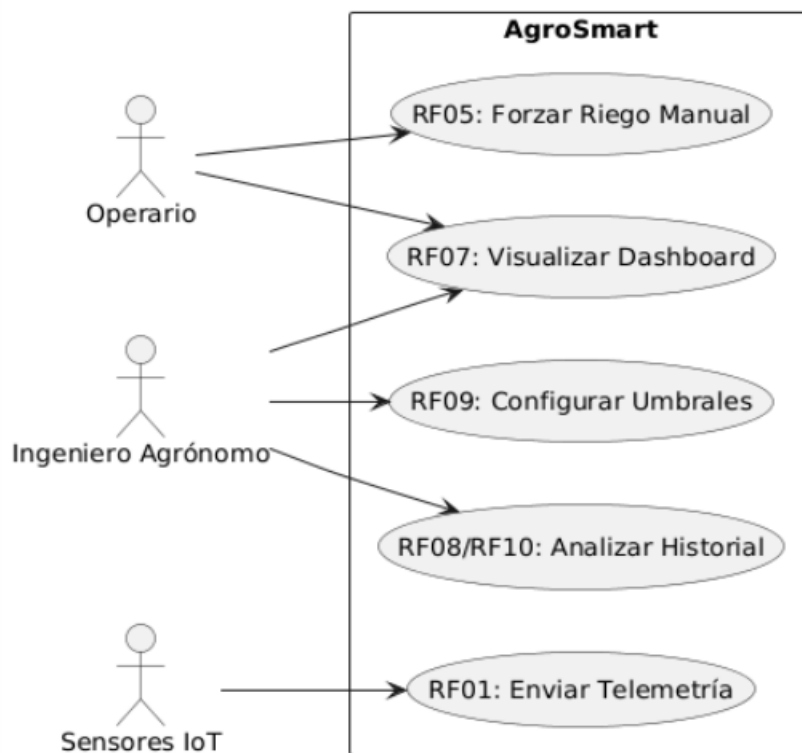
- **RNF01 Latencia IoT:** El comando de riego manual debe procesarse en el hardware en menos de 2 segundos.
- **RNF02 Disponibilidad:** El motor de reglas (backend) debe tener un uptime del 99.9%.
- **RNF03 Escalabilidad:** Soportar la conexión simultánea de hasta 1,000 nodos (sensores).
- **RNF04 Seguridad:** La comunicación IoT debe usar TLS 1.2 (MQTTs) y la web usar HTTPS.

Matriz de Trazabilidad Completa

ID Req.	Caso de Uso Vinculado	Módulo Arquitectónico	ID Prueba
RF01	CU01 - Enviar Datos	Módulo de Ingesta (Broker)	CP-IOT-01
RF02	CU02 - Crear Zonas	Gestión Agronómica	CP-ZON-01
RF03	CU03 - Evaluar Reglas	Motor de Decisiones	CP-RUL-01
RF04	CU04 - Consultar Clima	Integración Externa (API)	CP-EXT-01
RF05	CU05 - Riego Manual	Módulo de Control Remoto	CP-CMD-01
RF06	CU06 - Detectar Caídas	Sistema de Monitoreo	CP-MON-01
RF07	CU07 - Ver Dashboard	Interfaz de Usuario (UI)	CP-UI-01
RF08	CU08 - Consultar Gráficas	Módulo de Reportes	CP-REP-01
RF09	CU09 - Ajustar Umbral	Gestión Agronómica	CP-ZON-02
RF10	CU10 - Exportar Datos	Módulo de Reportes	CP-REP-02

ID Req.	Caso de Uso Vinculado	Módulo Arquitectónico	ID Prueba
RF11	CU11 - Acceso al Sistema	Seguridad y Autenticación	CP-SEC-01
RNF04	<i>Aplica a todo el flujo</i>	Infraestructura y Red	CP-SEC-02

Modelos de casos de uso:



2. Arquitectura de software

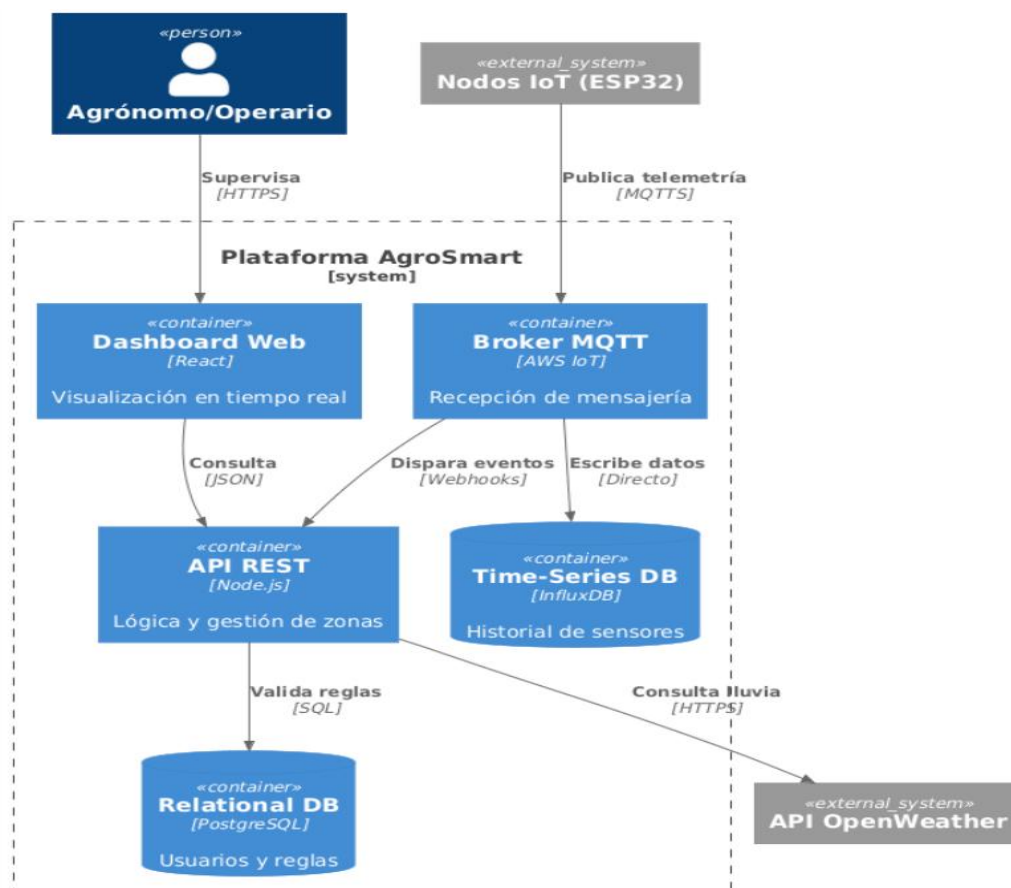
Documento SAD

Arquitectura Seleccionada: Orientada a Eventos y Cliente-Servidor.

- **Capa Edge:** Microcontroladores ESP32 comunicándose vía MQTT.
- **Capa Backend/Broker:** AWS IoT Core, Node.js.
- **Capa Persistencia:** InfluxDB (Telemetría) + PostgreSQL (Usuarios/Zonas).
- **Capa Presentación:** React.js + TailwindCSS.

Justificación: MQTT es ligero para redes rurales débiles. InfluxDB (Time-Series) optimiza la lectura/escritura de millones de datos generados por sensores, algo que colapsaría una BD relacional tradicional.

Diagramas de contenedores:

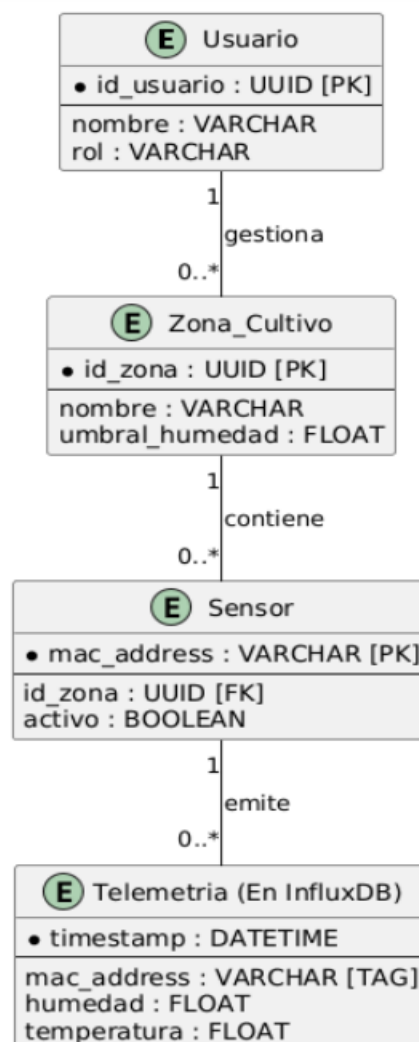


3. Diseño de software

Registro de Decisiones (ADR)

- **ADR-001 Protocolo de Hardware:** Decisión: Usar MQTT sobre HTTP.
Justificación: Ahorra batería en el hardware y permite conexión bidireccional continua (necesario para activar bombas).
- **ADR-002 Bases de Datos Duales:** Decisión: Usar InfluxDB y PostgreSQL.
Justificación: Separar el tráfico masivo de telemetría de los datos transaccionales de configuración para evitar cuellos de botella.

Diagrama de Entidad-Relación Lógico:



Contratos de API

Método	Endpoint	Descripción
GET	/api/v1/zonas/{id}/estado	Retorna última humedad y estado de la válvula.
POST	/api/v1/zonas/{id}/riego	Fuerza encendido manual (Envía payload al broker).
PUT	/api/v1/zonas/{id}/umbral	Modifica el % de humedad mínima de la zona.

Prototipos UI/UX

1. **Mapa Finca:** Vista satelital con polígonos dibujados; verde (sano), rojo (riego activo o alerta).
2. **Dashboard de Zona:** Velocímetros para humedad, termómetros, e interruptor ON/OFF para válvulas.

4. Construcción de software

Código fuente: Repositorio en GitHub estructurado en monorepo: /frontend, /backend, y /firmware (C++ para el ESP32).

Scripts de base de datos: Migraciones automatizadas usando Prisma para PostgreSQL, y scripts de políticas de retención de datos (Data Retention) para InfluxDB (borrar telemetría mayor a 1 año).

Documentación del desarrollador: README.md con instrucciones precisas para simular un sensor localmente mediante *Mosquitto MQTT* sin necesidad de hardware físico.

5. Pruebas de software

Plan de pruebas: Validar estrictamente la concurrencia del broker y las decisiones críticas de automatización.

Casos de prueba:

Código CP	Descripción	Resultado Esperado
CP-IOT-01	Enviar JSON simulado con humedad 20%.	Dato insertado en InfluxDB en <1s.
CP-RUL-01	Humedad cae al 20% (Umbral es 30%). Lluvia NO pronosticada.	Sistema publica comando "ON" a la bomba.
CP-RUL-02	Humedad cae al 20%. Lluvia SI pronosticada hoy.	Sistema NO publica comando (Ahorro de agua).
CP-CMD-01	Operario envía comando de Riego Manual.	Válvula se activa y sobrescribe reglas automáticas por 30 mins.

Reporte de cobertura: 85% en el backend (crítico en motor de reglas) y 70% en frontend (Jest/React Testing Library).

6. Operaciones de ingeniería del software

Scripts de IaC: Terraform para aprovisionar las instancias EC2, la VPC (Virtual Private Cloud) de AWS y la política de certificados para AWS IoT Core.

Pipelines de CI/CD: GitHub Actions. Al fusionar a main, se ejecutan pruebas, se compila la imagen Docker del backend y se despliega en AWS Elastic Beanstalk.

Tableros de monitoreo: Grafana conectado directamente a AWS CloudWatch para monitorear latencia de MQTT, memoria del servidor y uso de CPU.

7. Mantenimiento de software

Manual de operaciones: Si el estado del mapa está congelado, revisar logs de *AWS IoT* o reiniciar la pasarela (Gateway) del campo.

Plan de parches: Actualización automática mensual de dependencias Node.js, excepto parches de seguridad (CVEs) que se aplican el mismo día.

Registro de deuda técnica: 1. Migrar autenticación interna a un proveedor externo (Auth0). 2. Optimizar el *bundle* de React que carga el mapa satelital.

8. Gestión de configuración software

Estrategia de ramas: *GitFlow* (Ramas: main, develop, feature/*, hotfix/*).

Notas de liberación (Release Notes): v1.0.0 incluye Módulo Core de recepción, motor de reglas v1, y dashboard básico. Integración de clima queda para v1.1.

Inventario de artefactos: Archivo package.json para software, archivo platformio.ini para compilación del firmware de los sensores.

9. Gestión de ingeniería software

WBS/EDT: 1. Arquitectura Cloud; 2. Desarrollo Firmware; 3. Backend e Ingesta; 4. Interfaz Web; 5. Pruebas de Campo; 6. Despliegue.

Cronograma: Diseño (1 sem), Configuración Cloud/Hardware (2 sem), Desarrollo Software (3 sem), Pruebas físicas en invernadero (1 sem), Entrega (1 sem).

Matriz de Riesgos Detallada:

Riesgo	Causas	Probabilidad	Impacto	Estrategia de Mitigación
Falla de conexión en campo	Señal 3G/WiFi débil en la finca, clima adverso.	Alta	Alta	Hardware programado con rutina "Failsafe": si pierde red, riega por temporizador local.
Pérdida de datos	Caída del Broker MQTT o del servidor Backend.	Baja	Alta	Usar AWS IoT gestionado (alta disponibilidad) y QoS=1 en MQTT (garantiza entrega).
Saturación de Base de Datos	Demasiados sensores escribiendo al mismo tiempo.	Media	Media	Agrupamiento de envíos en el hardware (enviar promedios cada 5 min, no cada segundo).

Riesgo	Causas	Probabilidad	Impacto	Estrategia de Mitigación
Fallo en API de Clima	Límite de peticiones excedido o servidor externo caído.	Media	Baja	Si la API no responde, el motor ignora el clima y riega basado únicamente en la humedad del suelo.

10. Modelos y métodos de Ingeniería del software

Fichas de historias de usuario:

- **ID:** HU-02 - Configuración de Umbrales
- **Como:** Ingeniero Agrónomo.
- **Quiero:** Poder definir el límite mínimo de humedad para la Zona A.
- **Para:** Que el cultivo específico reciba agua automáticamente sin mi intervención.
- **Criterios de Aceptación:** 1. Ingresar valor entre 0-100%. 2. Ver indicador de "Guardado con éxito". 3. El motor de reglas debe actualizarse inmediatamente sin reiniciar el sistema.

11. Proceso de ingeniería software

Acuerdos de equipo:

- **DoR:** El modelo de payload MQTT del hardware está documentado antes de programar el backend.
- **DoD:** El código pasó revisión, las pruebas corren en verde y el hardware de prueba responde al comando en mesa.

Reporte de retrospectiva: Gran comunicación entre equipo de Hardware y Software; cuello de botella identificado en la configuración inicial de AWS.

12. Calidad del software

Reporte de análisis estático: Uso de *SonarQube* y *ESLint* configurado con reglas estrictas (Airbnb standard).

Métricas de calidad: Complejidad ciclomática < 10. Archivos de más de 300 líneas deben ser divididos. Deuda técnica medida menor a 4 días.

13. Seguridad de software

Modelo de amenazas:

- **Inyección de Comandos IoT:** Mitigado autenticando cada microcontrolador con certificados X.509 únicos incrustados en hardware. No se aceptan mensajes anónimos.
- **Interceptación de datos:** Todo el tráfico va sobre MQTTS / HTTPS (cifrado en tránsito).

Reporte SAST: Análisis automático en GitHub Actions detectando dependencias con vulnerabilidades conocidas (0 incidentes actuales).

14. Economía de Ingeniería de software

Análisis de ROI (Retorno de Inversión): Una hectárea mal regada en la región pierde hasta el 20% de rendimiento. Además, el costo de ineficiencia actual por supervisión manual y encendido físico de bombas es de aprox. 20 horas semanales (estimado a \$15.000 COP la hora = \$1.200.000 COP al mes en gastos operativos). Al automatizar el 90% del proceso, el ahorro mensual estimado es de **\$1.080.000 COP**, lo que cubre la inversión inicial en hardware en la primera etapa de la cosecha.

Presupuesto base mensual:

- **Hardware:** \$200.000 COP (Inversión inicial única por cada kit de ESP32 + Sensores de humedad/temperatura y relés).
- **Infraestructura Cloud (AWS):** ~\$60.000 COP mensuales (Instancia micro, base de datos y AWS IoT Core, manteniendo el consumo dentro de la capa gratuita académica).
- **Horas hombre (Desarrollo):** 160 horas. (Valorización simbólica para el proyecto académico estimada en \$2.400.000 COP si se cotizara como desarrollador Junior).